

NHS HEALTHSTORE · TECHNICAL REPORT

AI Architecture of HealthStore Alpha

The AI Commissioner Advisor and DTx Funding Finder —
what they are, how they work, and how they connect to the rest of the site.

Generated 28 July 2026

Source: `healthstore-alpha` repository

1. Executive summary

HealthStore Alpha has **one AI product surface**: the **AI Commissioner Advisor** — a consultative chat drawer for NHS commissioners — with a nested capability, the **DTx Funding Finder**, which researches live NHS funding opportunities for digital therapeutics.

Both features are built directly on the **OpenAI Node SDK** (Responses API, model `gpt-5.4`). There is no Vercel AI SDK, LangChain, embeddings, or vector database. The AI answers questions by calling **9 function tools** against the site's own catalogue data (apps, funding, conditions), plus OpenAI's built-in **web search** (locked to GB) for live information.

In one sentence: a logged-in commissioner chats with an OpenAI-powered advisor that reads the site's catalogue JSON and their organisation profile from Postgres, streams answers over SSE, and can spin up a nested LLM web-research pipeline to find and score NHS funding opportunities.

The two AI features at a glance

	AI Commissioner Advisor	DTx Funding Finder
What it does	Consultative chat: explore apps, costs, comparisons, conditions, implementation	Finds live NHS funding opportunities for deploying a digital therapeutic
Entry point	POST <code>/api/ai/chat</code> (SSE), UI drawer <code>AiAdvisorPanel</code>	Chat tool <code>find_dtx_funding</code> (also a standalone route, currently unwired)
Model	OpenAI <code>gpt-5.4</code> , medium reasoning, up to 8 tool rounds	OpenAI <code>gpt-5.4</code> (<code>OPENAI_MODEL</code> override), single call + web search
Data it reads	Catalogue JSON, static ICB profiles, Postgres <code>org_profiles</code>	Live web (GB), hard-coded regions/conditions/app list
Writes to DB?	No — stateless, client holds chat history	No
Key code	<code>lib/ai/*</code> , <code>app/api/ai/chat/route.ts</code> , <code>components/ai/*</code>	<code>lib/ai/funding/*</code>

Gating

The whole AI surface is feature-flagged: `NEXT_PUBLIC_AI_ADVISOR_ENABLED` must be truthy, auth must be enabled, and the user must be logged in. Otherwise the nav item is hidden, the panel is never mounted, and the API returns 404/401.

Environment variables

Variable	Role
<code>NEXT_PUBLIC_AI_ADVISOR_ENABLED</code>	Master feature flag for all AI surfaces
<code>OPENAI_API_KEY</code>	Required — OpenAI auth for both features
<code>OPENAI_MODEL</code>	Optional — overrides the Funding Finder model (chat is hardcoded to <code>gpt-5.4</code>)

DATABASE_URL	Indirect — enables org-profile personalisation; AI degrades to static profiles without it
--------------	---

2. System overview — how everything connects

The layout resolves the commissioner's profile server-side and mounts the chat drawer. The chat API orchestrates OpenAI with function tools; tools read the catalogue JSON, Postgres, or delegate to the Funding Finder, and results stream back to the panel as SSE events.

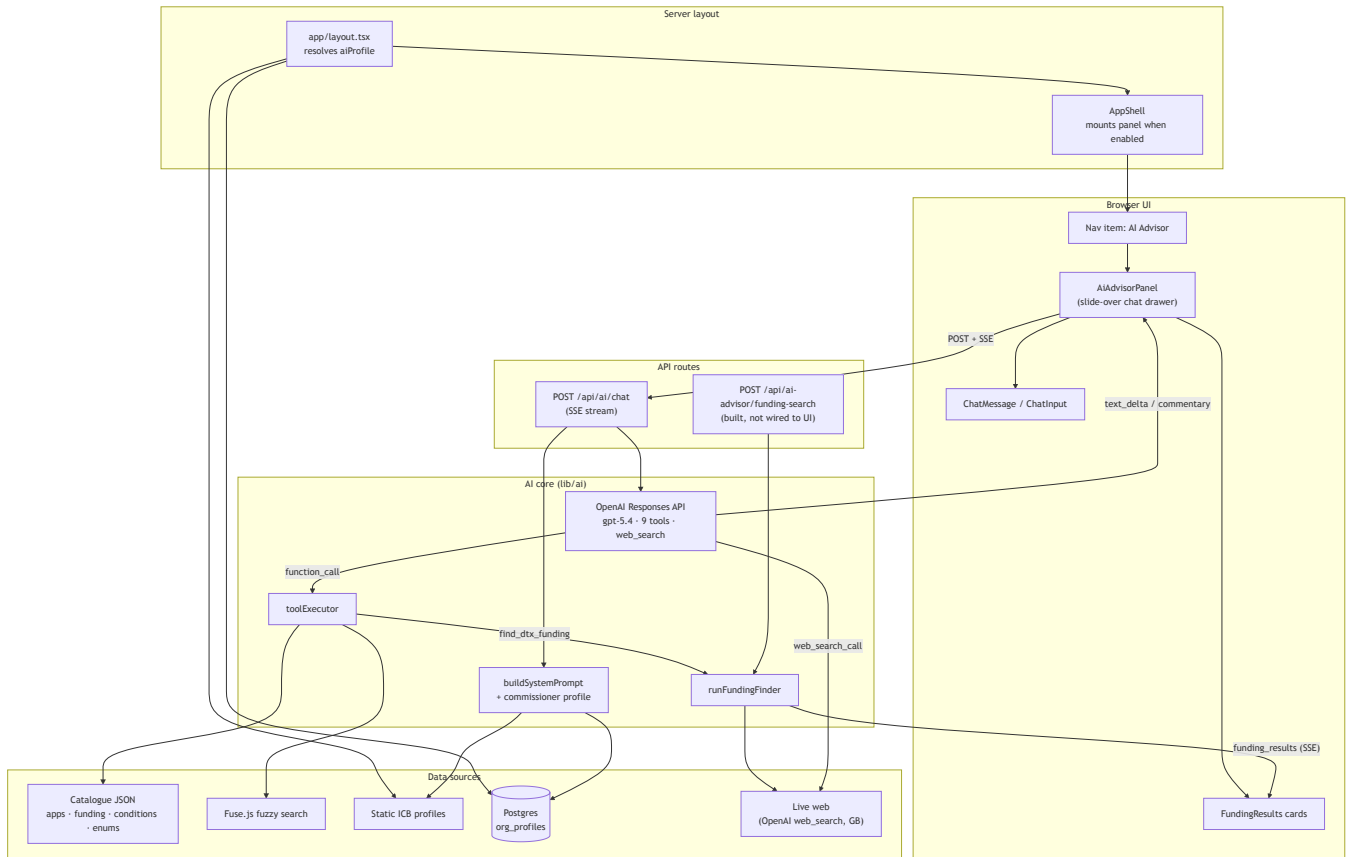


Figure 1 — Full AI architecture: UI, API routes, AI core, and data sources.

3. Chat request lifecycle

Each chat turn is a single stateless request. The server builds a hardened system prompt (identity lock, scope rules, NHS literacy, tool-use rules) plus the commissioner's ICB profile, then runs an OpenAI tool loop for up to 8 rounds. The client receives Server-Sent Events: `commentary` (progress), `text_delta` (answer), `funding_results`, and `done`. Suggestion chips are parsed from `[[...]]` markers in the assistant text.

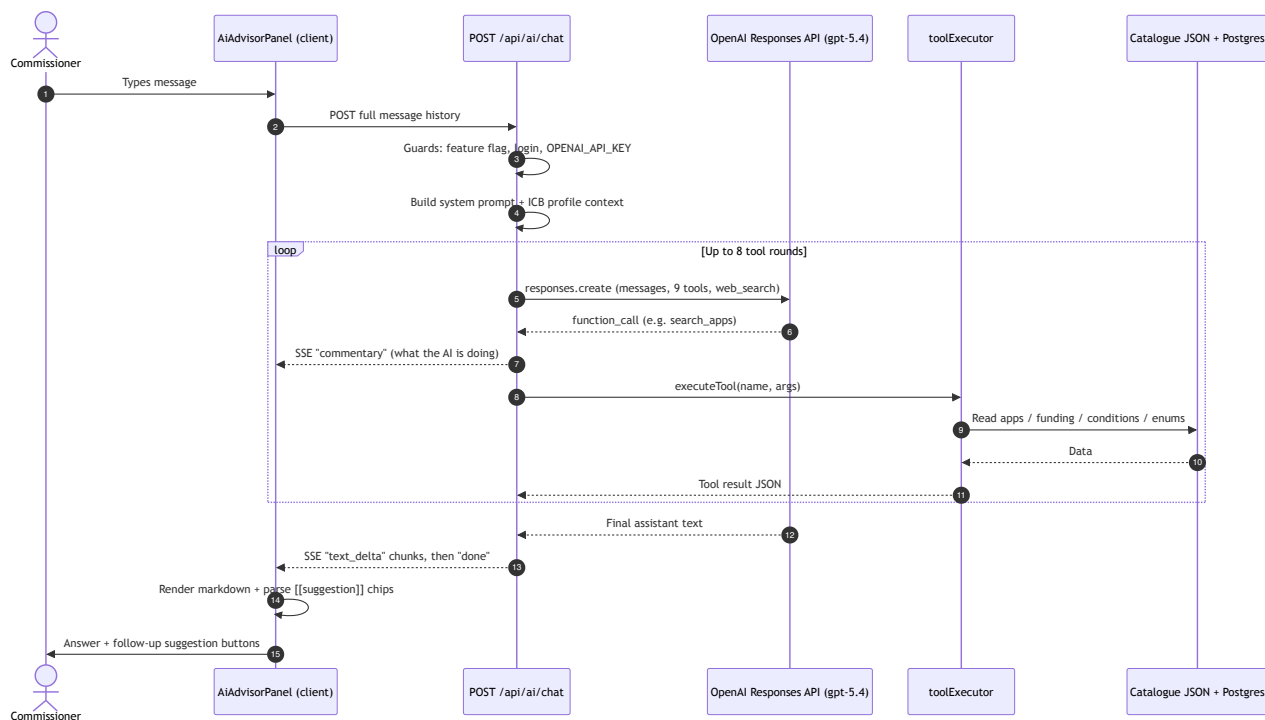


Figure 2 — One chat turn from keystroke to rendered answer.

4. The model's tools and their data sources

The model never touches data directly — it requests one of 9 function tools (defined in `lib/ai/tools.ts`, executed by `lib/ai/toolExecutor.ts`), plus OpenAI's built-in web search. Catalogue search uses Fuse.js fuzzy matching, not AI. The one special tool, `find_dtx_funding`, launches the nested Funding Finder pipeline.

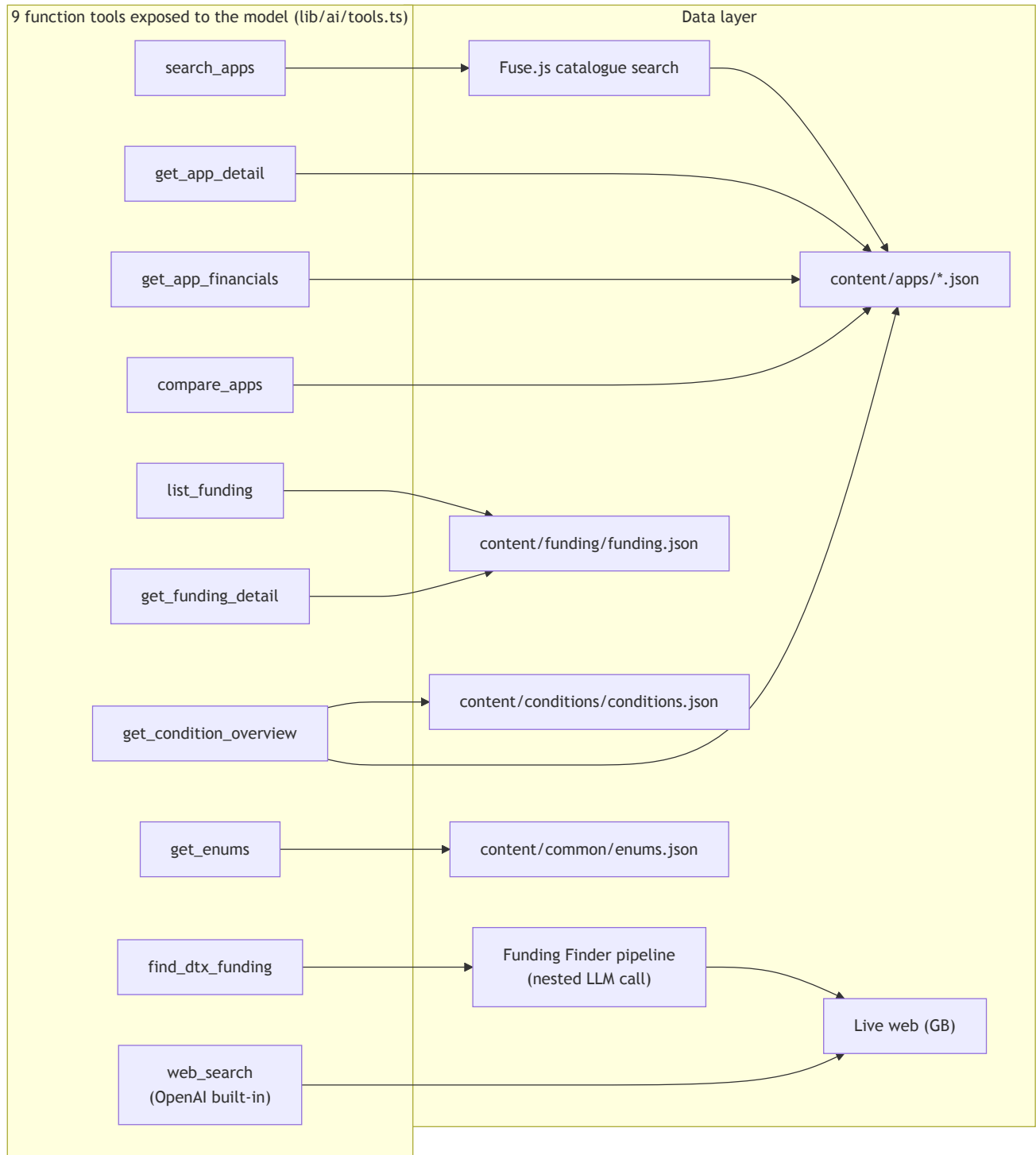


Figure 3 — Every tool the model can call, mapped to what it reads.

5. DTx Funding Finder pipeline

The Funding Finder makes **one** LLM call with web search enabled, asking for a JSON array of candidate NHS funding opportunities. Everything after that is deterministic application code: robust JSON extraction, eligibility filtering, 0–100 scoring, and ranking. Results stream into the chat as cards via the `funding_results` SSE event.

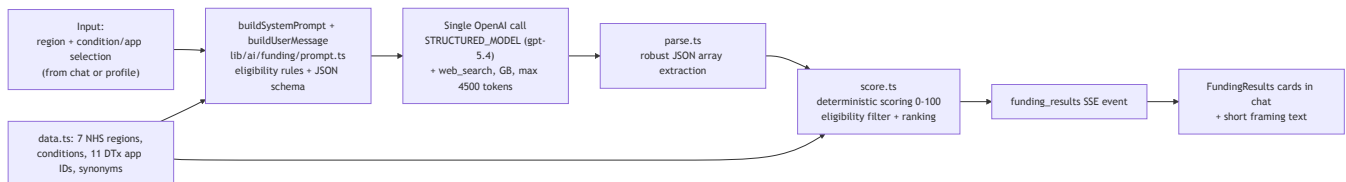


Figure 4 — LLM does research; code does parsing, scoring, and ranking.

Shortcut: the exact starter message *"I want to find NHS funding for deploying a digital therapeutic."* skips the chat loop and runs the finder immediately, using the region and condition focus from the commissioner's profile.

A standalone endpoint, `POST /api/ai-advisor/funding-search`, exposes the same pipeline directly (with a `FundingWizard` component), but neither is currently wired into the live UI — the conversational path is the active one.

6. Access gating and profile personalisation

Before any AI code runs, requests pass a chain of guards. Once through, the advisor is personalised: a static ICB profile (Shropshire, Cornwall, North East London, or West Yorkshire) is resolved from the session, then optionally overlaid with the organisation's own settings from the Postgres `org_profiles` table (written by the Org Settings page — the AI only ever reads it). If the database is unreachable, the AI gracefully falls back to the static profile.

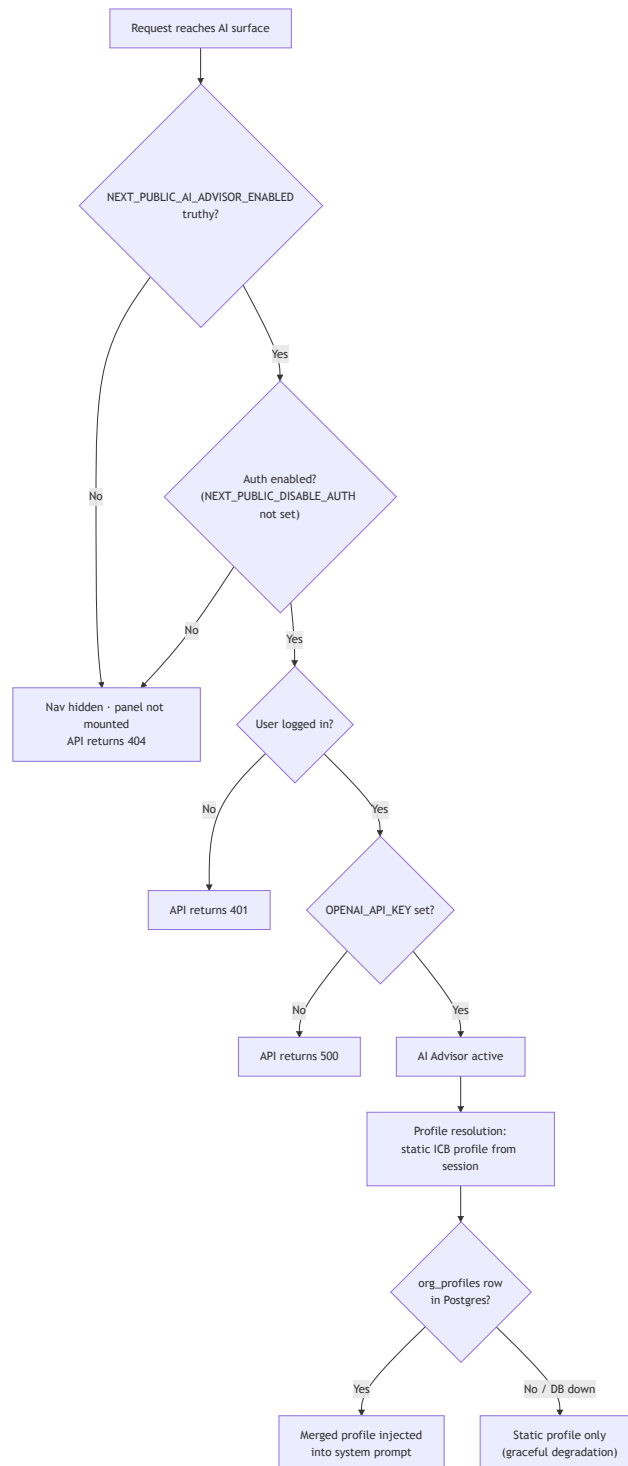


Figure 5 — Guard chain and profile resolution feeding the system prompt.

Key file reference

File	Role
lib/ai/config.ts	OpenAI client, model constants, reasoning effort, max tool rounds
lib/ai/systemPrompt.ts	Hardened system prompt + profile injection
lib/ai/tools.ts / toolExecutor.ts	Tool schemas and execution against the data layer
lib/ai/stream.ts	SSE event encoding (text_delta , commentary , funding_results , ...)
lib/ai/commissionerProfiles.ts	4 synthetic ICB profiles + session resolution
lib/ai/organisationProfileResolver.ts	Merges static profile with Postgres org_profiles
lib/ai/funding/*	Funding Finder: prompt, provider, parse, score, tests
app/api/ai/chat/route.ts	Auth-gated SSE chat endpoint (Node runtime, 300 s max)
components/ai/AiAdvisorPanel.tsx	Chat drawer UI and SSE consumer